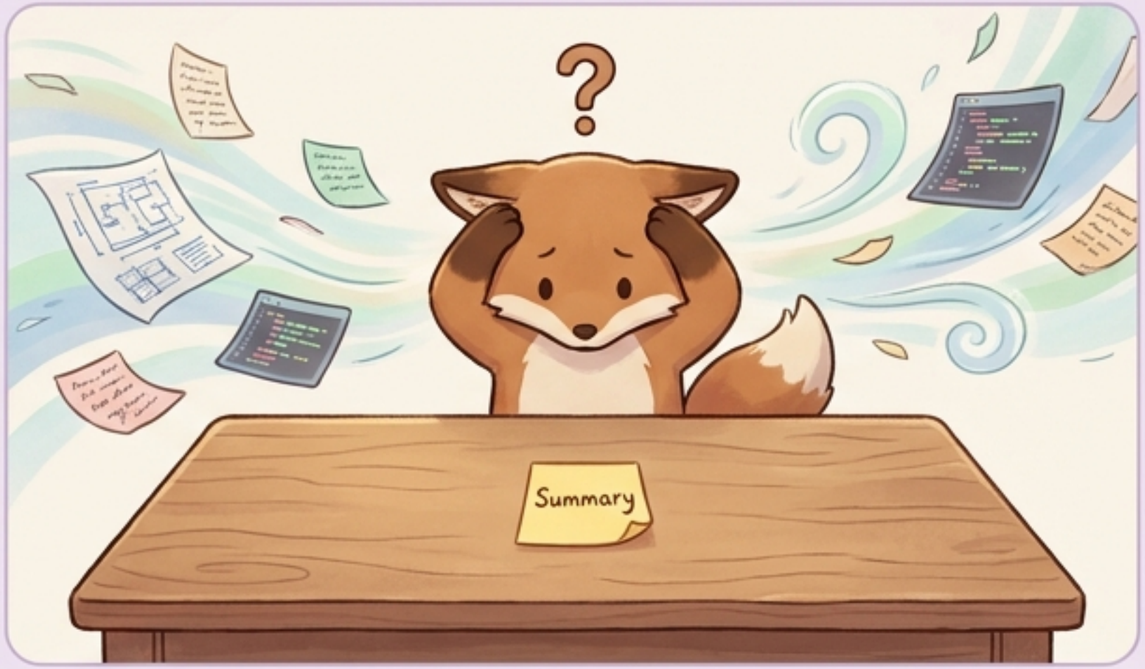
Endless Mode v1: Surviving the Memory Wipe

One markdown bottle file that lets AI coding sessions survive context compaction.

When sessions run long, **compaction** destroys the active context window

Before Compaction

After Compaction



Claude Code summarizes long conversations into a few paragraphs, throwing away original plans, exact wording, and trial histories. The session survives, but the memory doesn't. Users experience the assistant getting dumber mid-session.

Note: The summary cannot be replaced or suppressed by a plugin.

Compaction only destroys the context window—the real truth sits safely on disk

The Transcript

`~/.claude/projects/…/<sessionId>.jsonl`

Claude Code appends every user message, assistant message, and tool call. Compaction never touches it.

The `claude-mem` Database

`SQLite`

Real-time captures of observations, user prompts, and summaries, continuously written by background hooks.

Endless Mode reconstructs the session record and passes it up in a "bottle"

The Mechanism

At the start of every post-compact (or resumed) context, `claude-mem` writes a pure markdown file—the bottle—reconstructing the timeline from disk.

The Injection Instruction

We hand the model a simple pointer:
“Read this file and continue.”

Key Property

The bottle is a cache, derived on-demand and never maintained. No hook appends to it; it cannot drift.

Swapping heavy tool traffic for compressed observations shrinks the footprint by 10-20x

Conversational Spine (Verbatim)

User & assistant messages sourced from the transcript.
Result: Passed through untouched.

Tool Traffic (Observations)

Tool calls and results.
Result: Omitted entirely. Swapped for human-readable observations generated in real time.

Result: The model resumes from its own exact last words, without the bloat of raw tool output.

The dedicated actors bringing Endless Mode to life

Claude Code Hooks

Subprocesses that receive JSON payloads on stdin. Handle SessionStart and Stop events.

Rule: Hooks must never break the session (swallow errors, exit 0).

The Background Worker

An Express daemon running locally. Owns the DB and SDK agent.

Long work follows a queue-and-return shape to keep the active session moving instantly.

The SQLite Storage

~/.claude-mem/claude-mem.db.

A highly structured sorter storing session states, verbatim prompts, and generated observations.

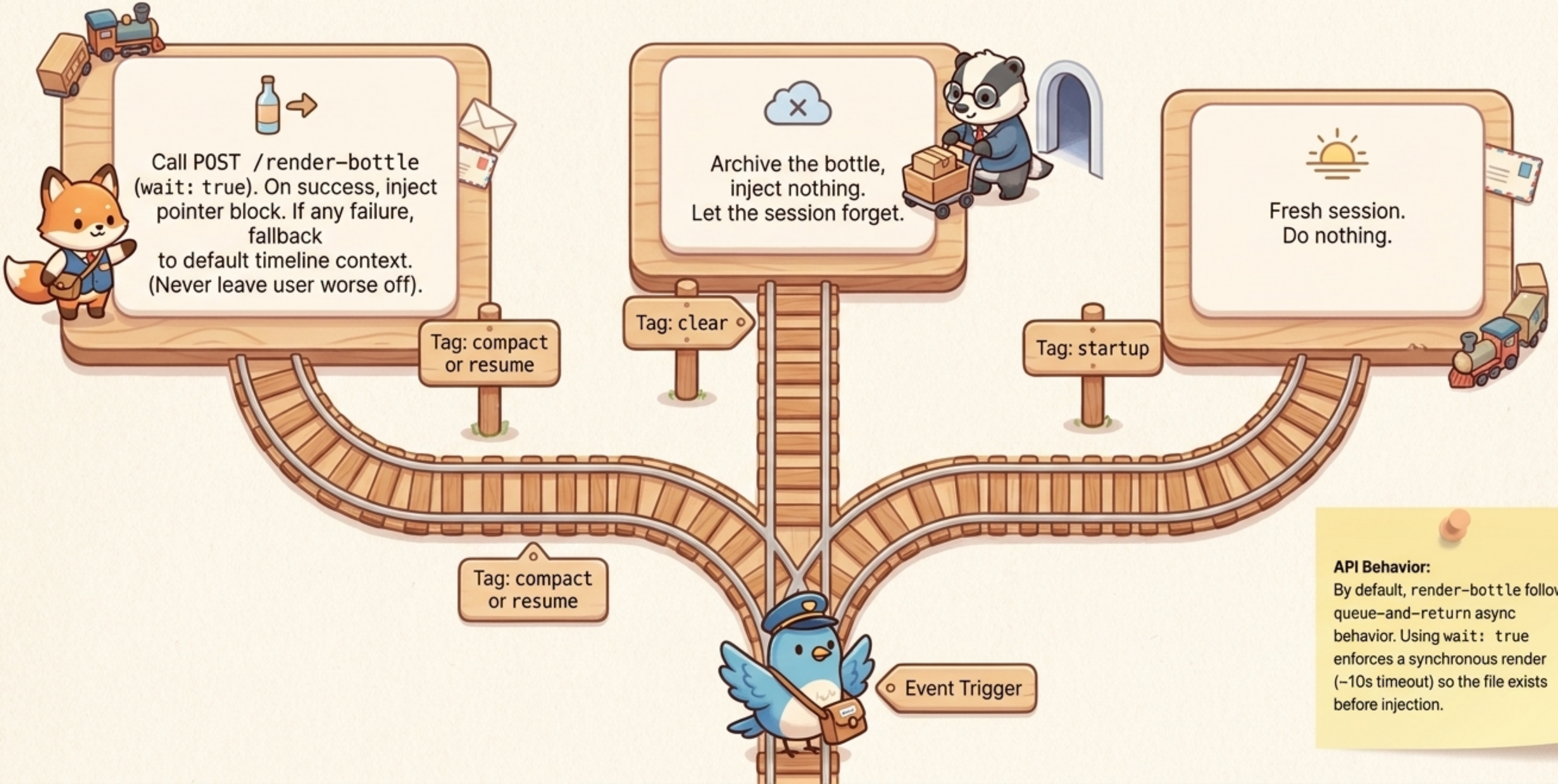
🔒 **Privacy Contract:** Every piece of text written to disk passes through `stripMemoryTags` to scrub <private> data and host-injected noise.

One stable key unlocks all session memory, regardless of worker restarts

Component 1: The `BottleRenderer` pure function

Output: Atomic write to disk. (Writes temp sibling, `rename()` over target to prevent corruption).

Component 2 & 3: The API Route and `SessionStart` Hook



API Behavior:
By default, `render-bottle` follows `queue-and-return` async behavior. Using `wait: true` enforces a synchronous render (~10s timeout) so the file exists before injection.

The injection is just a pointer, not the payload

The Pointer Strategy

The payload is a ~20 line prompt telling the model to “Read” the file. This lands as a tool result—the exact category of tokens that the next compaction will evict first. This prevents bottles from accumulating inside themselves.

```
... "role": "tool", "content": "Reading file..." ...
```

The Load-Bearing Instruction

Instruction injected: “Do not repeat your final message.” Without this exact line, models reliably resend their final message or redo already finished work.

```
"Do not repeat your final message."
```

The Failsafe

A “Current task” line acts as insurance for the rare context that skips the file Read.

```
Current task: Review document
```

Degraded Mode triggers when transcripts vanish

🔍 The Trigger

Claude Code prunes old transcripts (~30 days).
If missing, `BottleRenderer` relies entirely on SQLite.

🛠️ The Reconstructed Bottle

- `user_prompts`: Rendered verbatim (truncated at 4,000 chars).
- `observations`: Positioned by `prompt_number`.
- `session_summaries`: Rendered as visibly generated blocks (e.g., `> Session summary`).

THE GOLDEN RULE: Never put fake words in the Assistant's mouth! Summaries are never rendered under an Assistant heading. Assistant messages are unrecoverable without a transcript.

Defining the boundary: What we are intentionally skipping

Idea	Reason
No PreCompact Hook	The transcript and DB survive compaction automatically. There is nothing to rescue. Render lazily at `SessionStart`.
No blocking on LLM observation queues	Freshest tool events may lack observations. That's fine; the verbatim final assistant tail narrative. Blocking buys almost nothing.
No bottle-state table	The file is the state; the injection is an idempotent pointer. Double-fires are harmless.
No byte cursors / incremental parsing	Full re-parse matches the baseline cost. Renders are pure functions, so concurrent renders are safe last-writer-wins.

The 7 Essential Tests for Endless Mode

Compact mid-turn
- Incomplete turn → bottle captures assistant text verbatim.

Privacy scrubbing
- `<private>` content never makes it into the rendered .md.

Degraded render
- Delete transcript → correct headers inject, summaries quote correctly, truncation markers present.

Worker down
- Exit 0 → falls back safely to today's timeline context.

`/clear`
- Archives bottle, injects nothing.

Double SessionStart
- Dual injections do not corrupt the context.

E2E validation
- Model reads bottle, continues work seamlessly, does not repeat final message.

The Horizon: Candidates for v2

Historical CLI

`npx claude-mem bottle <id>` to render any historical session and resume it fresh (powered by Degraded Mode).

```
$ npx claude-mem bottle
> _
```

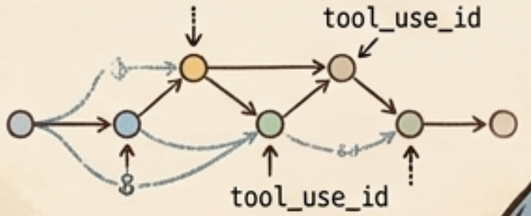
Leg Roll-Up

Collapse oldest spans to observation IDs for marathon sessions, saving tokens while keeping lossless references.

COLLAPSE

Micro-Anchoring

Per-tool-call observation threading (`tool_use_id` columns) for ultimate placement precision.



Live Diary

Server-runtime parity for render-on-Stop, keeping a real-time human-readable session file updated every turn.

Advanced mechanical bottle upgrades

Gear system

Advanced gear system

Expanded mechanical storage vault